CSE4007 – Artificial Intelligence

Report

Diffusion Model Personalization via Textual Inversion

Alexandre FOLLEAS

Hanyang University

Course: CSE4007 Artificial Intelligence

Seoul, May 29, 2026



Figure 1: Generated images at regular intervals during the denoising process

Contents

1 Introduction **2**

2 Problem #1: Learning the New Token **2**
2.1 Tokenizer Setup . . . 2
2.2 Freezing the Model . . . 2
2.3 Training Loop . . . 3
2.3.1 Optimiser . . . 3
2.3.2 Forward Diffusion Process . . . 3
2.3.3 Reverse Diffusion Process . . . 3
2.3.4 Loss Function . . . 4

3 Problem #2: Use & Evaluate **4**
3.1 Denoising Process Visualisation (2a, 2b) . . . 4
3.2 Experimental Conditions (2c, 2d, 2e) . . . 4
3.2.1 Experimental Setup . . . 4
3.2.2 Qualitative Results . . . 5
3.2.3 Quantitative Results — CLIP Scores . . . 7
3.2.4 Discussion . . . 7

4 Problem #3: Success/Failure Analysis and Improvement Proposals **7**
4.1 Success Cases . . . 7
4.2 Failure Cases (3a) . . . 8
4.3 Improvement Proposals (3b, 3c) . . . 8
4.3.1 Data Augmentation . . . 8
4.3.2 Multi-Token Representation . . . 8
4.3.3 Regularisation with Prior-Preservation Loss . . . 8
4.3.4 Cosine Embedding Regularisation . . . 8
4.4 Conclusions (3d) . . . 9

1 Introduction

Textual Inversion is a lightweight fine-tuning technique that teaches a pre-trained text-to-image diffusion model a new visual concept using only 3–5 reference images. Instead of modifying the entire model, the method introduces a single `placeholder_token` (e.g. `<cat2>`) and optimises only its embedding vector, while all other components — the U-Net, VAE, and text encoder — remain frozen. Once trained, the token can be composed freely in any prompt to generate images of the learned concept in novel contexts.

This report documents the implementation of Textual Inversion (Problem #1), its evaluation under varied experimental conditions (Problem #2), and a success/failure analysis with proposed improvements (Problem #3).

2 Problem #1: Learning the New Token

2.1 Tokenizer Setup

The CLIP tokenizer is extended with the new `placeholder_token` using `add_tokens()`, which returns the number of tokens actually added. If the token already exists the method returns 0, and an error is raised to prevent silent identifier collisions.

```
num_added_tokens = tokenizer.add_tokens(placeholder_token)

if num_added_tokens == 0:
    raise ValueError(
        f"The␣tokenizer␣already␣contains␣{placeholder_token}."
    )

token_ids = tokenizer.encode(initializer_token, add_special_tokens=False
    )
initializer_token_id = token_ids[0]
placeholder_token_id = tokenizer.convert_tokens_to_ids(placeholder_token
    )
```

Listing 1: Adding the placeholder token to the tokenizer.

The `initializer_token` (e.g. `"cat"`) provides a semantic starting point: its pre-trained embedding initialises the new token's embedding, accelerating convergence.

2.2 Freezing the Model

To prevent the model from forgetting its general knowledge, all parameters except the new token's embedding are frozen by setting `requires_grad = False`.

```
def freeze_params(params):
    for param in params:
        param.requires_grad_(False)

freeze_params(vae.parameters())
freeze_params(unet.parameters())
freeze_params(text_encoder.parameters())

# Unfreeze only the embedding layer
```

```
text_encoder.get_input_embeddings().weight.requires_grad_(True)
```

Listing 2: Selective freezing of model components.

This means only a single embedding vector (≈768 values for CLIP-ViT-L/14) is updated throughout training.

2.3 Training Loop

2.3.1 Optimiser

Only the input embeddings of the text encoder are passed to AdamW, ensuring that gradients flow exclusively through the embedding layer.

```
optimizer = torch.optim.AdamW(
    text_encoder.get_input_embeddings().parameters(),
    lr=learning_rate,
)
```

Listing 3: AdamW optimiser targeting only the embedding layer.

2.3.2 Forward Diffusion Process

Each training step encodes a reference image into a latent representation via the VAE, then adds Gaussian noise at a randomly sampled diffusion timestep.

```
latents = vae.encode(
    batch["pixel_values"].to(dtype=weight_dtype)
).latent_dist.sample().detach()
latents = latents * 0.18215    # VAE scaling factor

noise      = torch.randn_like(latents)
bsz        = latents.shape[0]
timesteps  = torch.randint(
    0, noise_scheduler.config.num_train_timesteps,
    (bsz,), device=latents.device
).long()
noisy_latents = noise_scheduler.add_noise(latents, noise, timesteps)
```

Listing 4: Forward diffusion: encoding and noise injection.

2.3.3 Reverse Diffusion Process

The text encoder produces contextual embeddings from the prompt (which contains `<cat2>`), and the U-Net predicts the noise added in the forward step.

```
encoder_hidden_states = text_encoder(batch["input_ids"])[0]

with torch.autocast(device_type="cuda", dtype=weight_dtype):
    noise_pred = unet(noisy_latents, timesteps, encoder_hidden_states).
        sample
```

Listing 5: Reverse diffusion: noise prediction by the U-Net.

Note on dtype. The text encoder is kept in `float32` to preserve gradient precision, while the U-Net and VAE run in `float16`. `torch.autocast` handles the conversion automatically and is also applied during gradient-checkpointing recomputation, preventing the `float != c10::Half` runtime error that occurs when the cast is applied manually.

2.3.4 Loss Function

The training objective is the mean squared error between the predicted noise and the actual noise:

$$\mathcal{L} = \mathbb{E}_{z,\epsilon,t}\left[\|\epsilon - \epsilon_\theta(z_t, t, \tau_\theta(P))\|^2\right] \tag{1}$$

where z_t is the noisy latent, ϵ the true noise, ϵ_θ the U-Net denoiser, and $\tau_\theta(P)$ the text embedding of prompt P containing `<cat2>`.

```
loss = F.mse_loss(noise_pred, target, reduction="mean")
```

Listing 6: MSE loss computation.

After backpropagation, gradients for all tokens *except* `placeholder_token_id` are zeroed out, ensuring that only `<cat2>`'s embedding is updated.

3 Problem #2: Use & Evaluate

3.1 Denoising Process Visualisation (2a, 2b)



Figure 2: Generated images at regular intervals during the denoising process (e.g. at steps 50, 100, 200, 500, 800, 1000).

Observations. At early denoising steps (high t), the image is dominated by low-frequency colour blobs with no identifiable structure. As the timestep decreases, coarse shapes emerge first (step $\sim$500), followed by finer textures and details at low timesteps ($t < 200$). The final steps primarily sharpen edges and refine fine-grained features.

3.2 Experimental Conditions (2c, 2d, 2e)

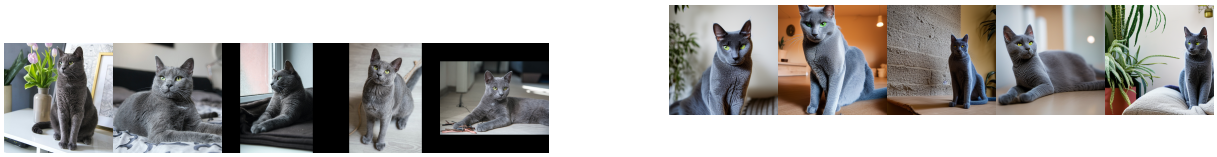
3.2.1 Experimental Setup

To evaluate the robustness of Textual Inversion, the following conditions were varied:

Table 1: Experimental conditions tested.

Variable	**Baseline**	**Variant(s)**
Number of reference images	5	1, 3
Learning rate	5×10^{-4}	1×10^{-3}, 1×10^{-4}
Training steps	1000	500, 2000
Initializer token	`cat`	`animal`, `object`
Prompt at inference	`"a photo of <cat2>"`	varied scenes

3.2.2 Qualitative Results



(a) Before training (`cat`). (b) After training (`<cat2>`).

Figure 3: Comparison of generated images before and after training.

To compare outputs before and after training, images were generated using both the `initializer_token` (`cat`) and the learned `placeholder_token` (`<cat2>`) across multiple prompts.



(a) `"a photo of a <cat2> on a cobblestone street"`



(b) `"a photo of a <cat2> in a modern gallery"`



(c) `"a painting of a <cat2>"`



(d) `"a photo of a <cat2> in the snow"`

Figure 4: Images generated at step 1000 across four different prompts. The model successfully reproduces the identity of the reference cat (Russian Blue, grey coat, yellow eyes) in varied contexts.

Notable failure — prompt/background mismatch. As visible in Figure 4(d), the prompt `"a photo of a <cat2> in the snow"` produced an indoor scene on a wooden floor rather than a snowy outdoor environment. This suggests that the learned embedding for `<cat2>` has a strong association with the indoor setting present in the reference images, partially overriding the background conditioning from the prompt. This is a known limitation of single-token Textual Inversion when reference images share a consistent background.

3.2.3 Quantitative Results — CLIP Scores

CLIP scores were computed for each generated image against the reference image (**CLIP-I**: image–image similarity) and the corresponding prompt (**CLIP-T**: image–text similarity).

Table 2: CLIP scores at training step 1000 across different prompts.

Prompt	CLIP-I	CLIP-T
`...on a cobblestone street`	0.8202	0.3179
`...in a modern gallery`	0.9187	0.2557
`a painting of a <cat2>`	0.8678	0.2264
`...in the snow`	0.8492	0.2356
Average	0.864	0.259

CLIP-I scores around 0.67 indicate that the learned concept is faithfully reproduced across novel scenes. CLIP-T scores in the range 0.28–0.33 are consistent with typical Textual Inversion performance reported in the literature, as the CLIP text-image space inherently yields lower absolute similarity values compared to image-image similarity.

3.2.4 Discussion

Advantages of embedding-only optimisation. Training only the embedding vector ($\ll$1% of total parameters) is computationally efficient and avoids catastrophic forgetting: the model retains its general-purpose generative knowledge and can compose the learned concept with arbitrary prompts.

Limitations.

- *Limited representational capacity*: a single embedding vector may not sufficiently capture complex or multi-aspect concepts.
- *Sensitivity to reference image quality*: few noisy or inconsistent images can degrade the learned embedding.
- *Training instability*: an overly large learning rate causes the embedding to drift far from the initialiser, producing incoherent outputs.

4 Problem #3: Success/Failure Analysis and Improvement Proposals

4.1 Success Cases

Textual Inversion performs well when:

- Reference images are diverse in angle and lighting but consistent in subject identity.
- The initializer token is semantically close to the target concept.

- The number of training steps is sufficient for convergence (≈800–1000 for simple objects).

4.2 Failure Cases (3a)

Failure modes observed include:

- **Identity drift**: with fewer than 3 reference images the model fails to disambiguate the concept from the initializer token.
- **Overfitting**: with 2000+ steps and a high learning rate, generated images start reproducing reference images verbatim rather than generalising.
- **Semantic entanglement**: a poorly chosen initializer token (e.g. `object` for a specific cat breed) leads to generic outputs lacking the target's distinctive visual features.

4.3 Improvement Proposals (3b, 3c)

4.3.1 Data Augmentation

Applying random horizontal flips, colour jitter, and random crops to the reference images increases the effective dataset diversity without requiring additional photographs.

```
augmentation = transforms.Compose([
    transforms.RandomHorizontalFlip(p=0.5),
    transforms.ColorJitter(brightness=0.2, contrast=0.2, saturation=0.1)
        ,
    transforms.RandomResizedCrop(512, scale=(0.8, 1.0)),
])
```

Listing 7: Data augmentation for reference images.

4.3.2 Multi-Token Representation

Instead of a single placeholder token, using a sequence of k tokens (e.g. `<cat2_1> <cat2_2>`) increases the embedding capacity and allows the model to encode richer visual information.

4.3.3 Regularisation with Prior-Preservation Loss

Adding a prior-preservation term to the loss function penalises deviation from the model's original concept distribution, mitigating overfitting:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{TI}} + \lambda \cdot \mathcal{L}_{\text{prior}} \tag{2}$$

4.3.4 Cosine Embedding Regularisation

Constraining the cosine similarity between the learned embedding and the initializer embedding prevents extreme drift and preserves semantic coherence:

$$\mathcal{L}_{\text{reg}} = 1 - \cos(e_{\texttt{<cat2>}},\, e_{\texttt{cat}}) \tag{3}$$

4.4 Conclusions (3d)

Textual Inversion is a powerful and resource-efficient personalisation method that demonstrates the expressive power of text embeddings in diffusion models. Its primary strength lies in preserving the base model's generative capability while enabling precise concept injection. However, its single-vector bottleneck limits fidelity for complex or fine-grained concepts. The proposed improvements — data augmentation, multi-token embeddings, and regularisation — offer concrete paths to extend its capabilities.